Aula 01:
Ferramentas Básica EDA

Linux, Git, Shell Script, TCL, Synopsys EDAs p/ **DV**

RESIDÊNCIA EM MICROELETRÔNICA - POLO UFCG

Sumário

- Linux (review)
- Git
- Shell Script
- EDA DV Synopsis
- Tcl
- Dúvidas sobre CI-EXPERT

10%Teoria + 90%Prática

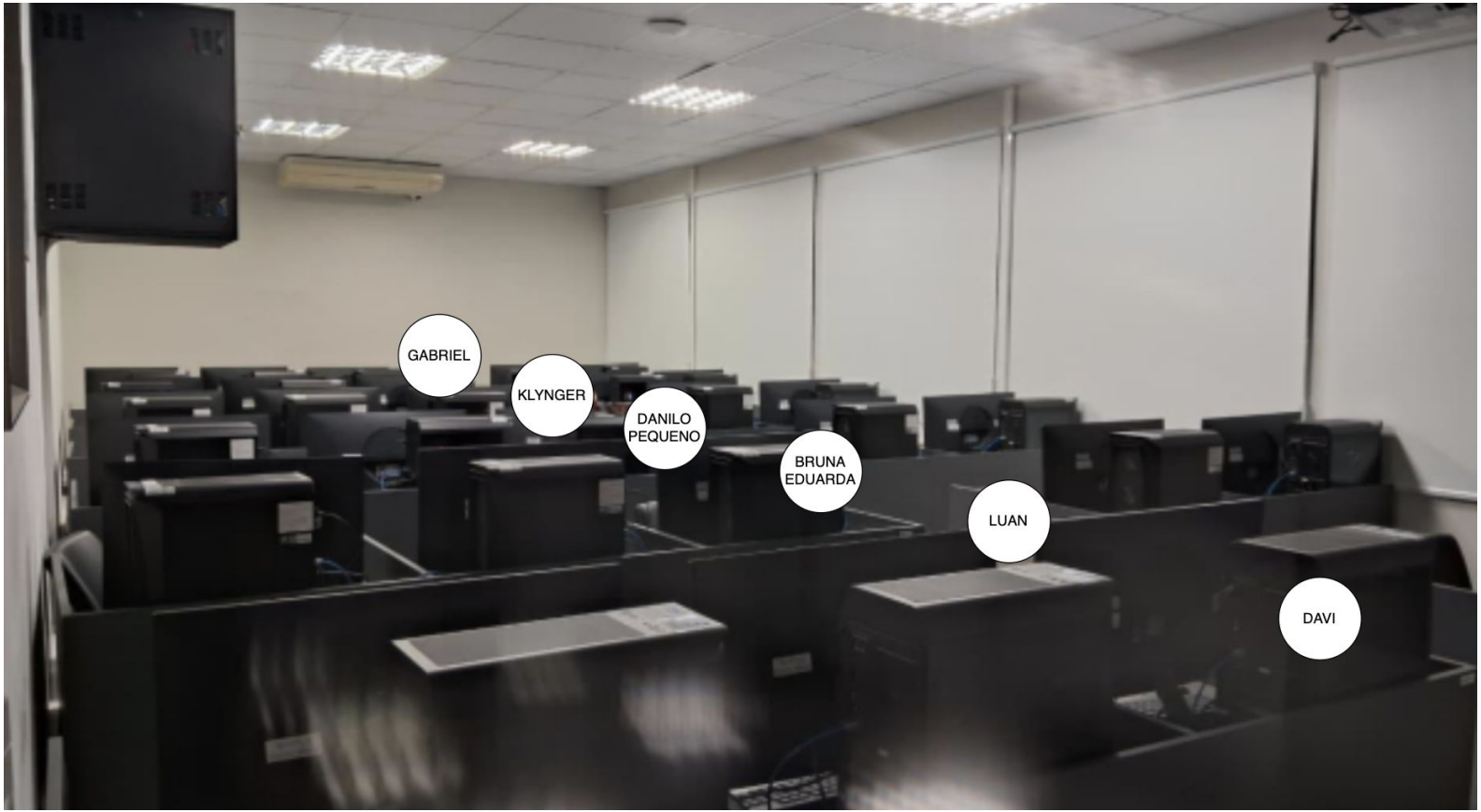
monitores

Dinâmica da Aula / Hands-On / Básico

- **Linux (review)**
  - Relembrar alguns comandos na prática + Dúvidas
- **Git**
  - Entender os conceitos enquanto faz + Dúvidas
- **Shell Script**
  - "Projetinho" pronto para vocês explorarem + Dúvidas
- **EDA DV Synopsis**
  - Usar "Projetinho" para fazer 2 exercícios em SystemVerilog **=D**
- **Tcl**
  - Hello World!

- **Dúvidas sobre CI-EXPERT**

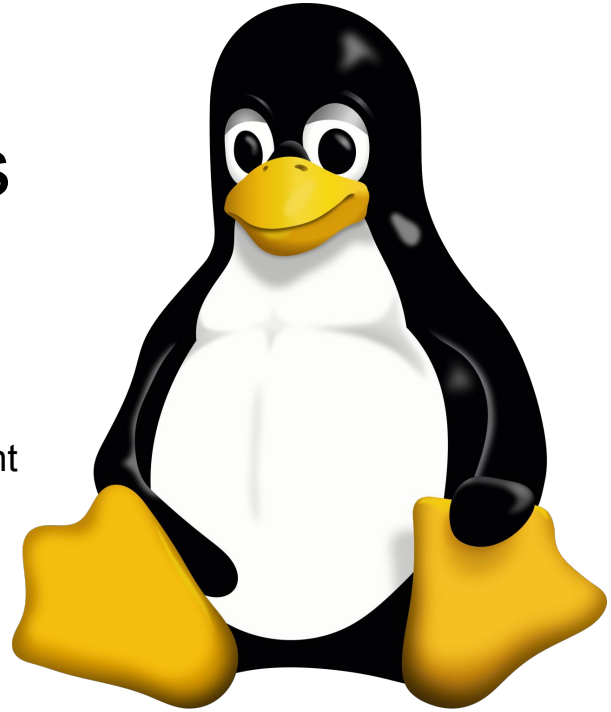
Monitores?



Linux

- Revisando comandos básicos
  - ls, cd, mkdir, cp, cat, touch,
  - echo, grep, alias, pwd

globally search for a regular expression and print



Linux

- Revisando comandos básicos
  - ls, cd, mkdir, cp, cat, touch,
  - echo, grep, alias, pwd

    globally search for a regular expression and print
- Atalhos Ctrl+R

Linux

- Revisando comandos básicos
  - ls, cd, mkdir, cp, cat, touch,
  - echo, grep, alias, pwd

    globally search for a regular expression and print

- Atalhos Ctrl+R

- Dúvidas?

Introdução

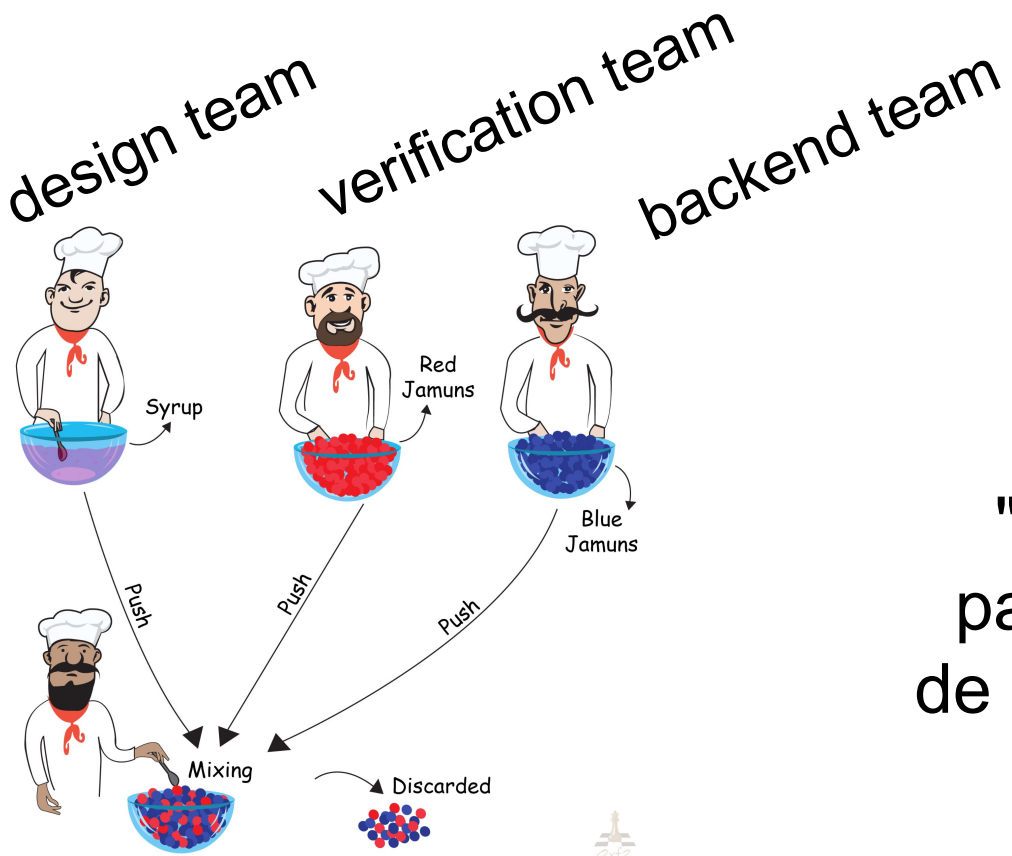
- Imagine que vc precisa administrar o trabalho de várias pessoas...

Introdução

- Imagine que vc precisa administrar o trabalho de várias pessoas...

- Onde vc vai centralizar o trabalho de cada um?
- E se o que você copiou já está desatualizado?
  - Voce vai pedir?
  - Se o desenvolvedor não lembrar?

Introdução



perspectiva de equipe
"projeto ASIC"

"todo mundo fazendo uma parte, vai misturando, depois de um tempo termina, elimina o que não usa e pronto"

main (for client / GDS)

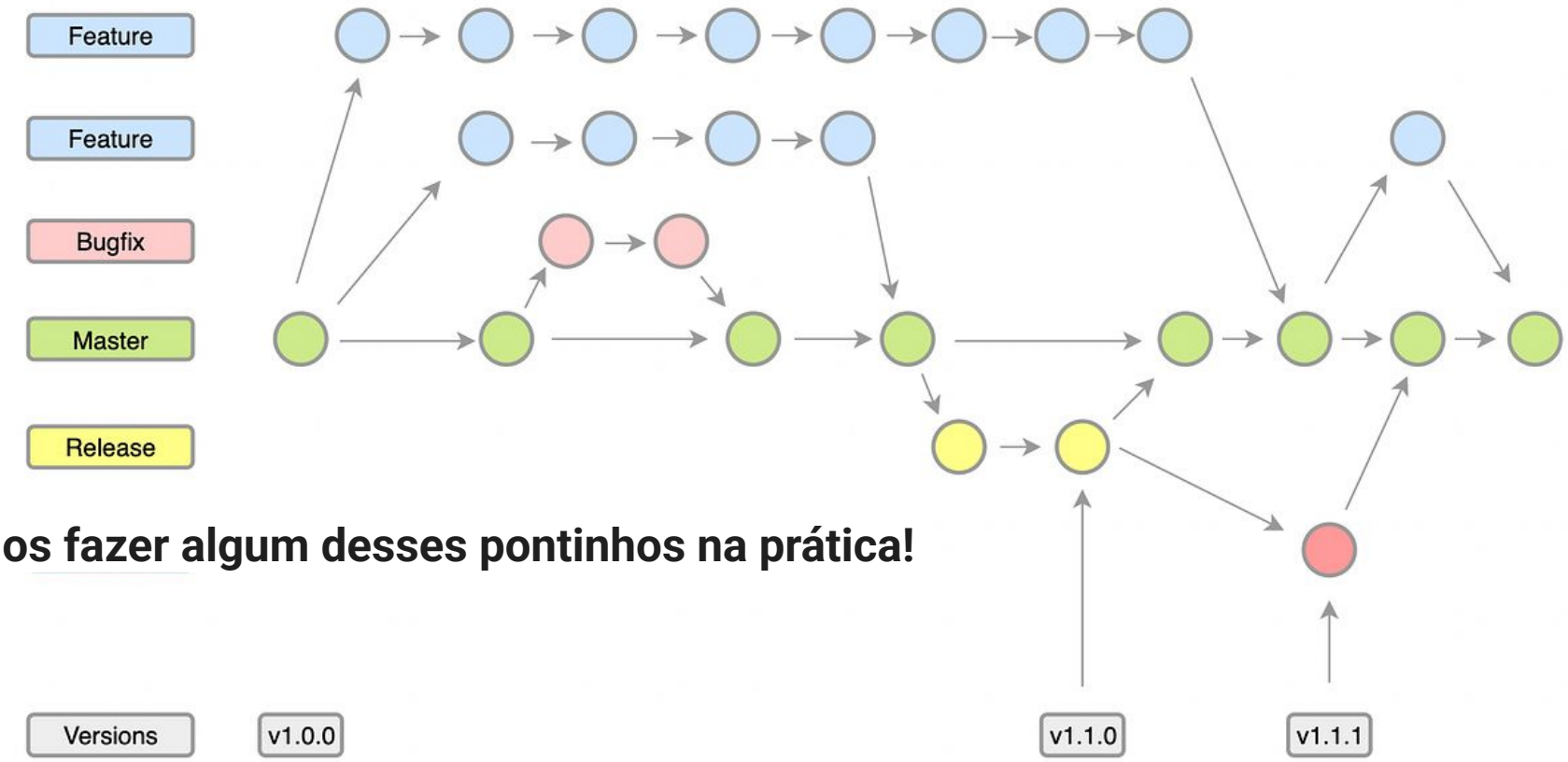
Introdução

- O que é o Git?

O Git é uma ferramenta usada para controlar e registrar as mudanças feitas nos arquivos de um projeto, como códigos de descrição de hardware (por exemplo, VHDL ou SystemVerilog), scripts de simulação, documentação, etc.

Ele mantém um histórico das modificações, permitindo voltar para versões anteriores do projeto, comparar alterações e **facilitar o trabalho em equipe**, onde várias pessoas podem desenvolver e testar circuitos digitais sem perder o controle das versões do projeto.

git Como Funciona?



Vamos fazer algum desses pontinhos na prática!

git Como Funciona?

- Primeiro passo é saber se o git está instalado

```
$ which git
$ # or
$ git -v
```

git Como Funciona?

- Definir quem é você:

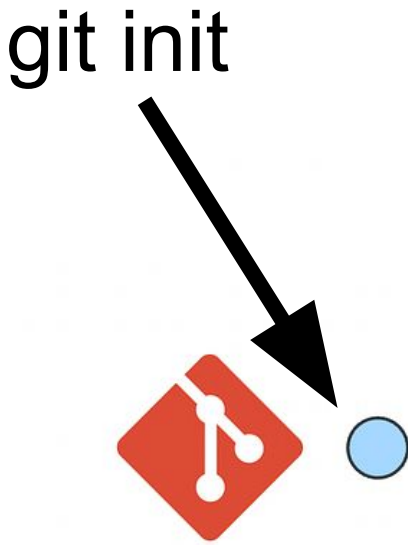
```
$ git config --global user.name "NAME LASTNAME"
$ git config --global user.email "name.lastname@virtus-cc.ufcg.edu.br"
```

git Como Funciona?

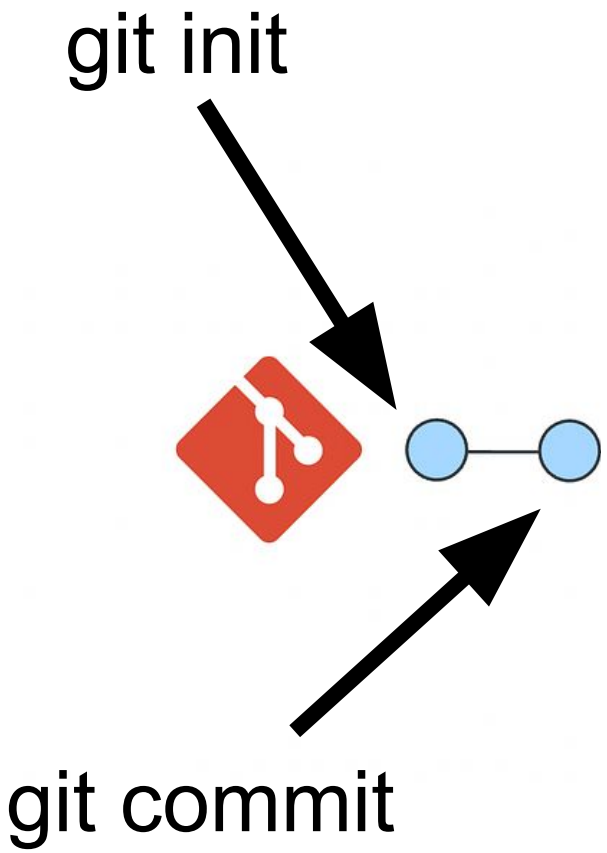
- Iniciar o repositório (o bowl)

```
$ mkdir -p workspace/labs/s1
$ cd workspace/labs/s1
```

git Como Funciona?



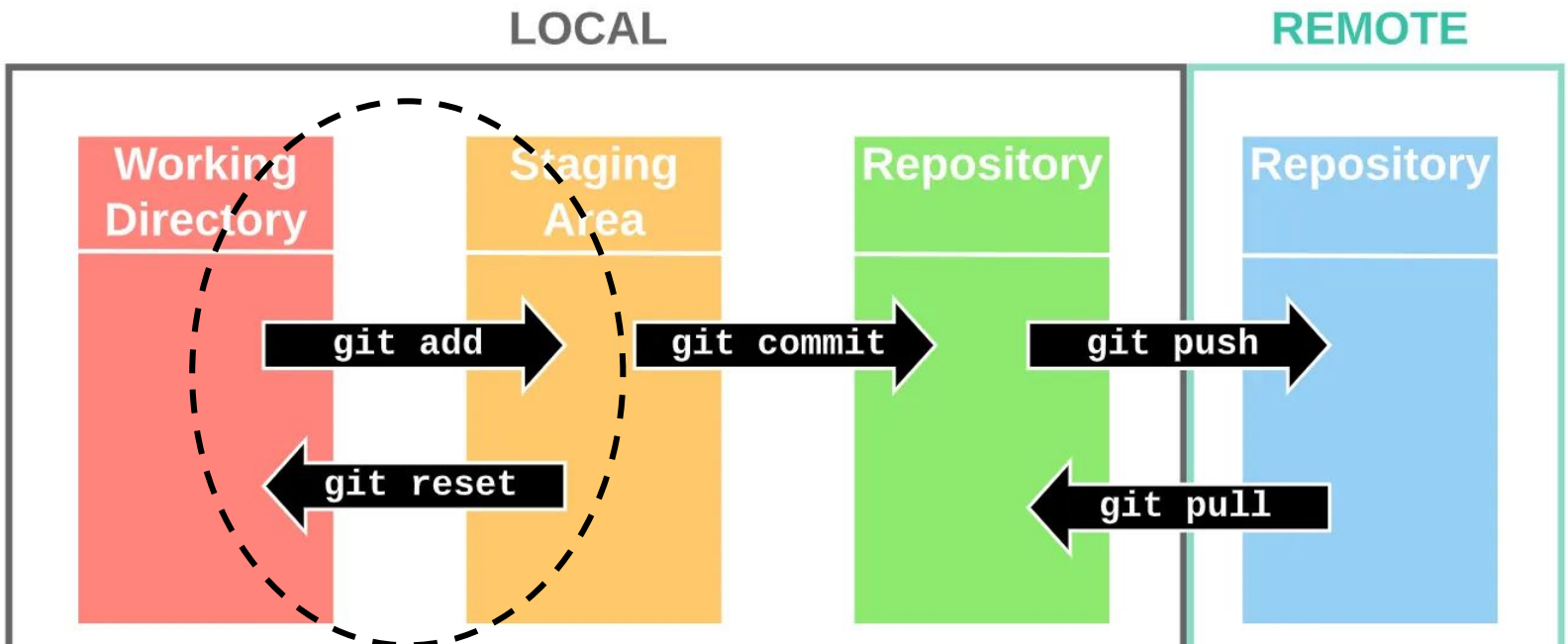
git Como Funciona?



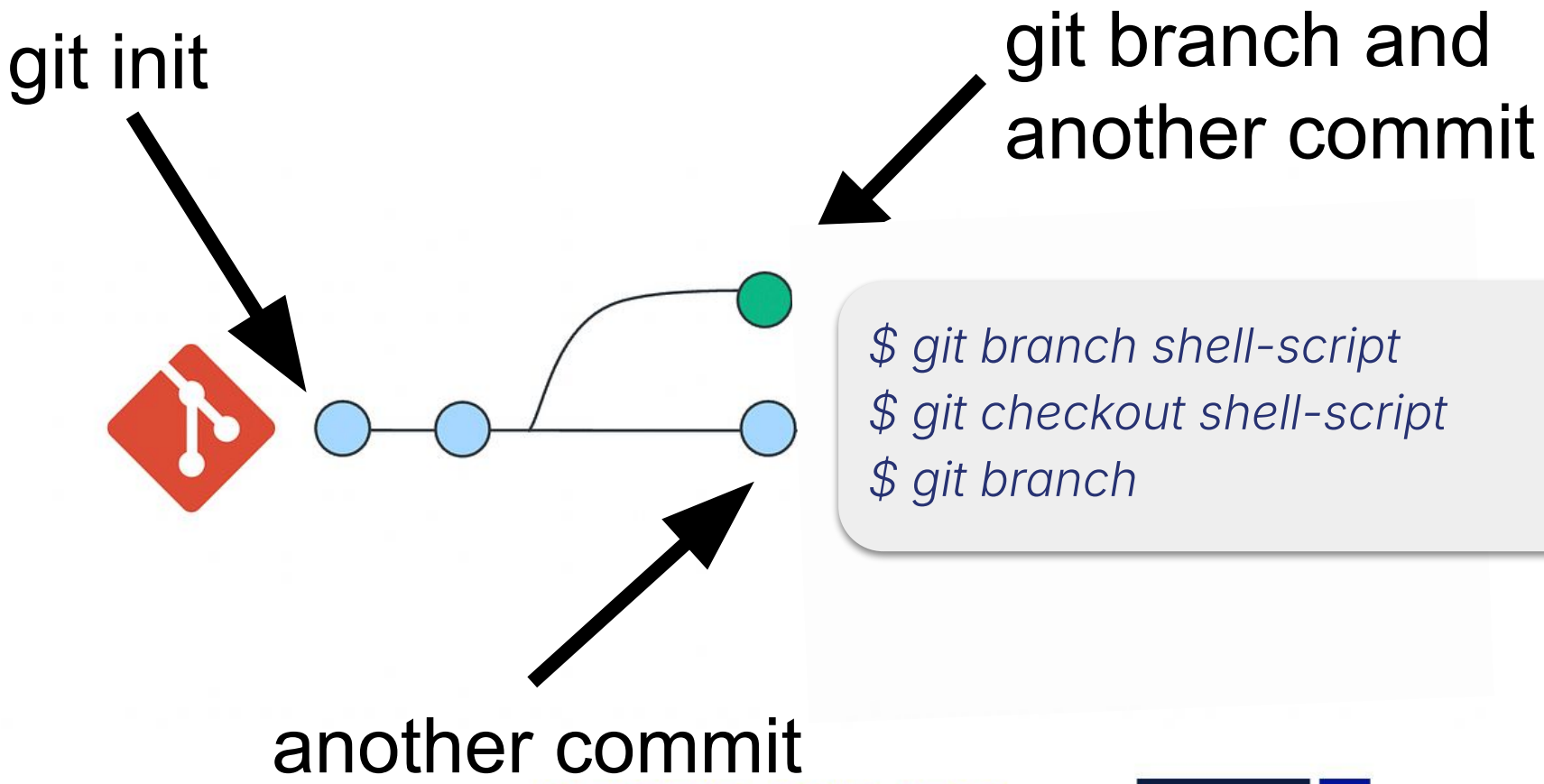

```
$ echo 'First commit' > README.md
$ git add README.md
$ git status
$ git reset
$ git add README.md
$ git commit -m "Initial commit"
```

git Como Funciona?

- Entendendo o Staging Area

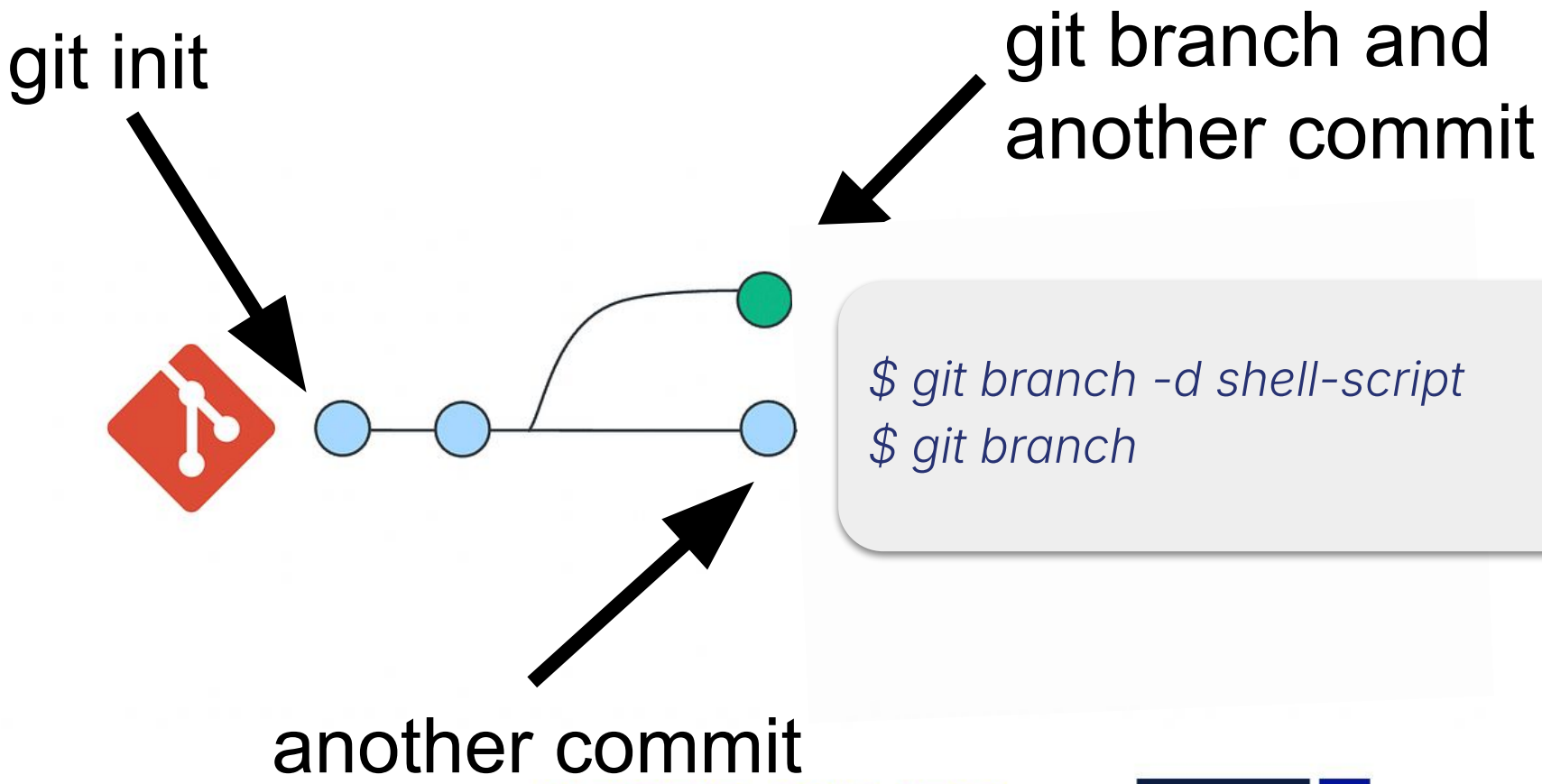


git Como Funciona?



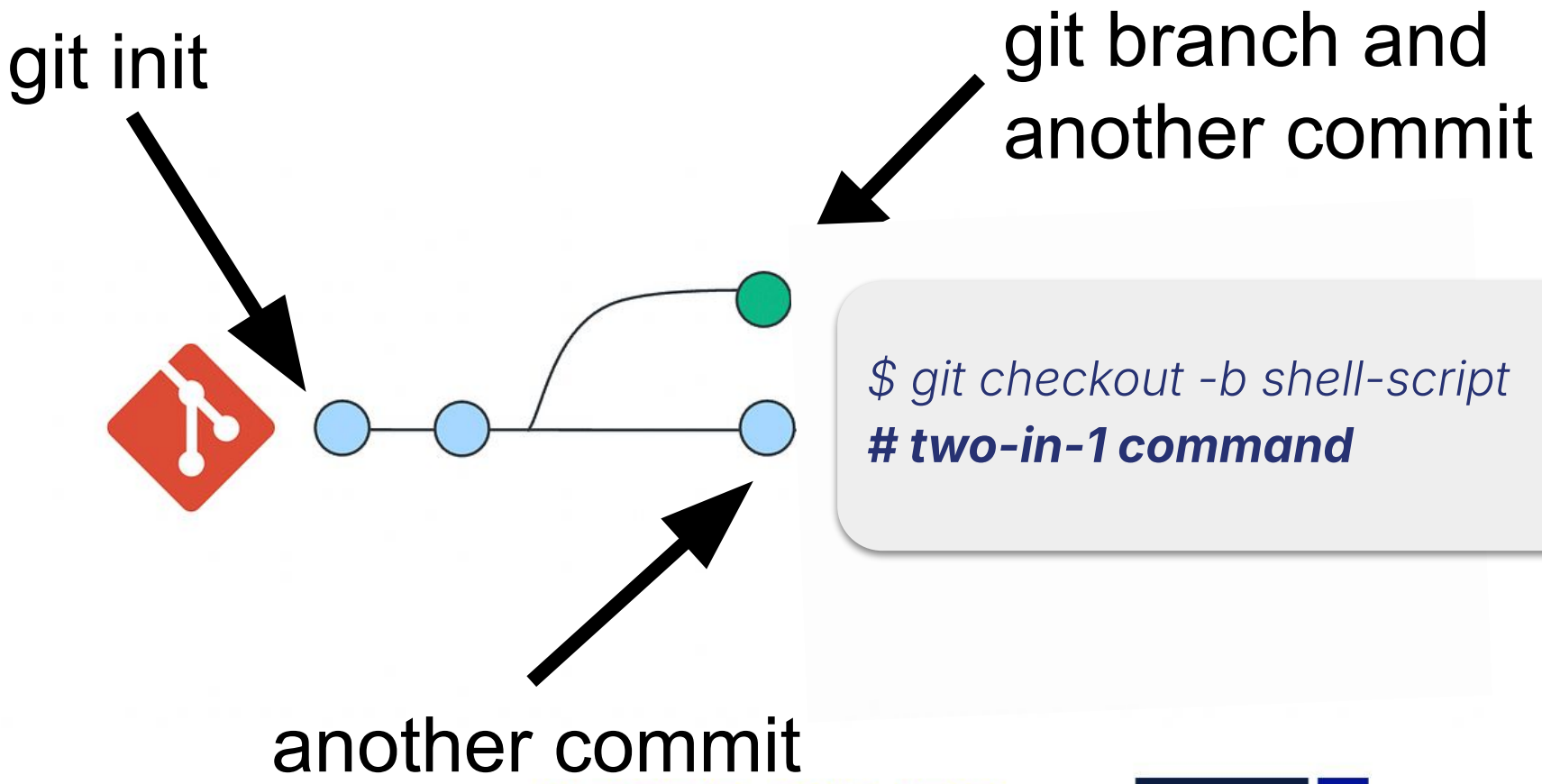

```
$ git branch shell-script
$ git checkout shell-script
$ git branch
```

git Como Funciona?



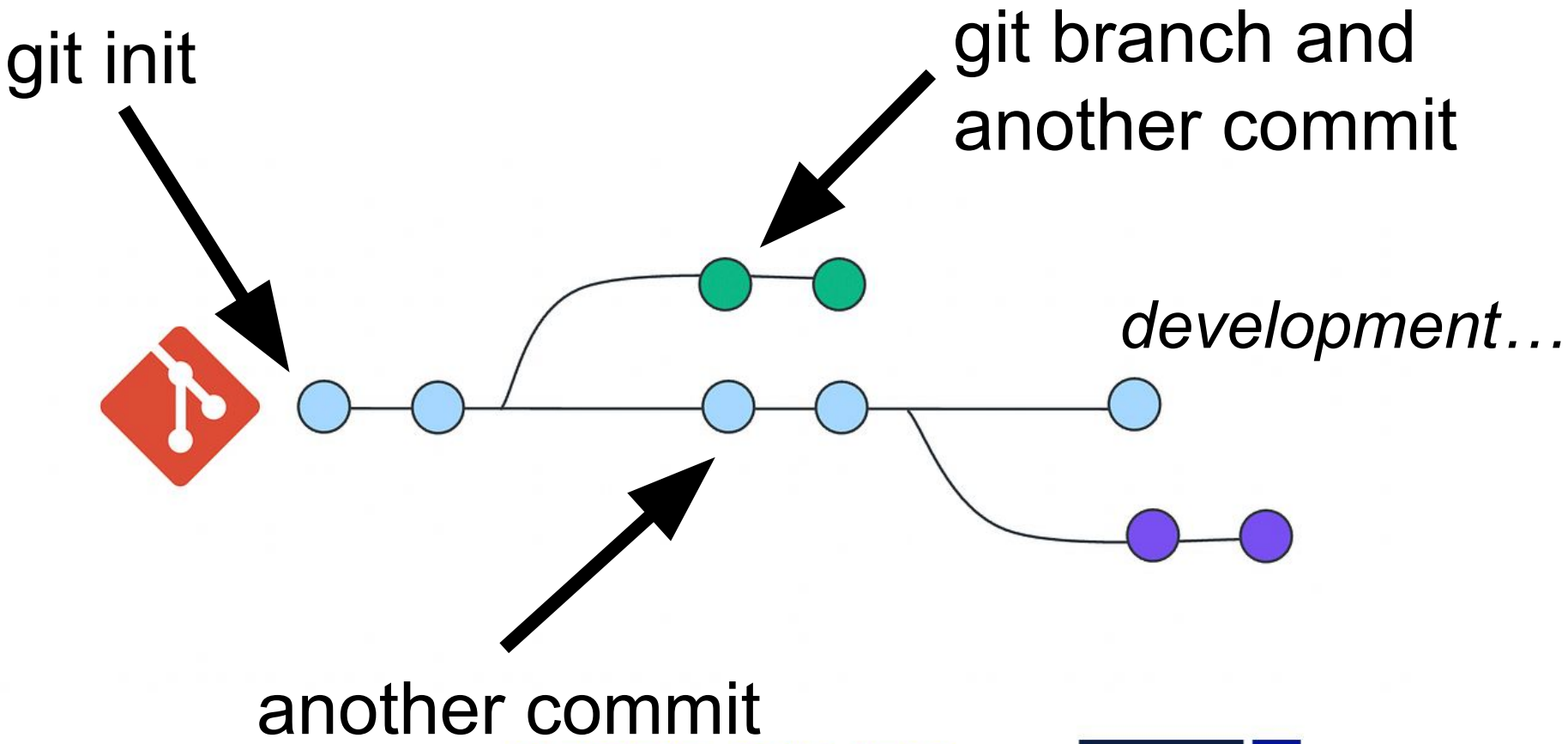

```
$ git branch -d shell-script
$ git branch
```

git Como Funciona?

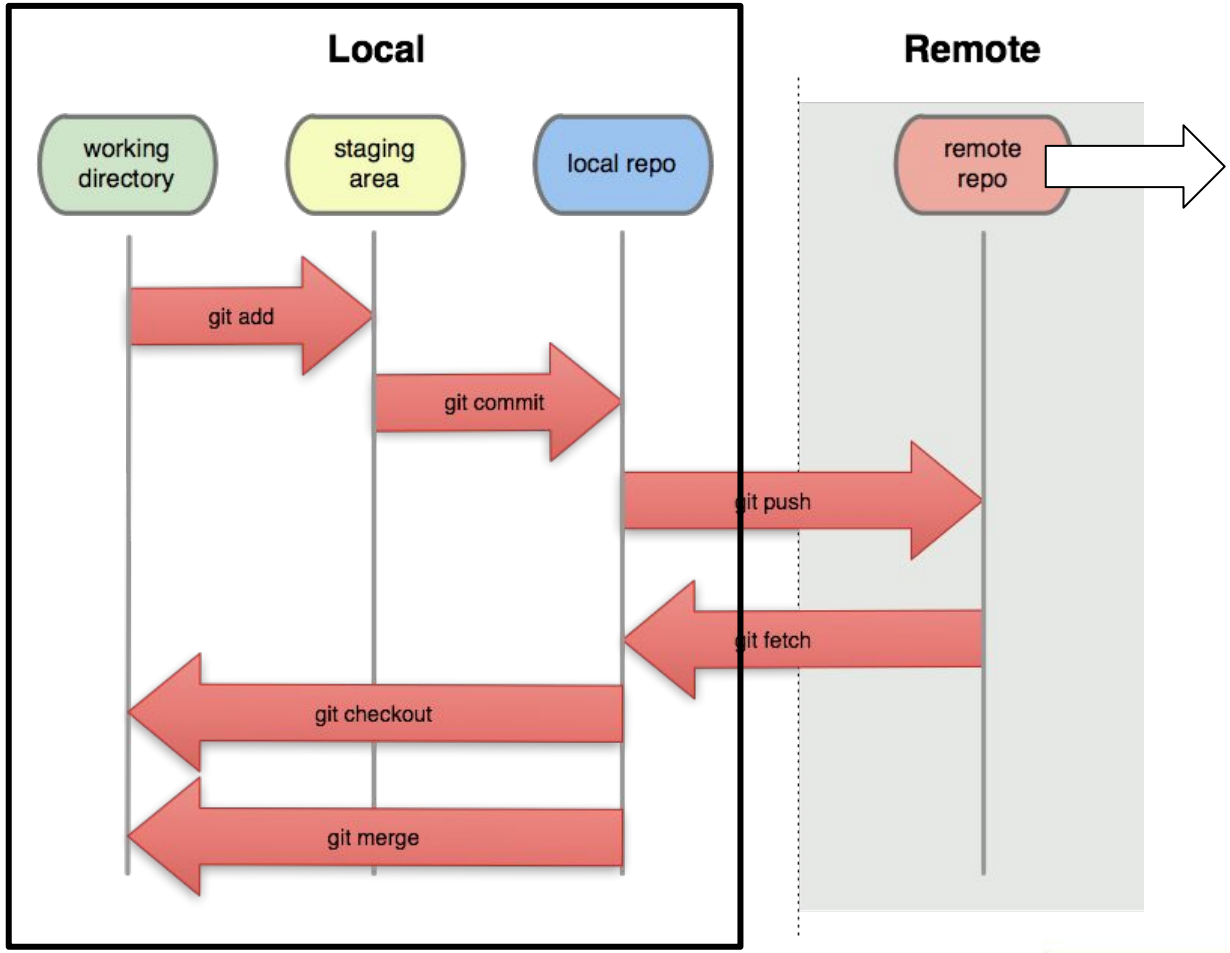



```
$ git checkout -b shell-script
# two-in-1 command
```

git Como Funciona?



git Como Funciona?



pode ser o GitHub, GitLab, um servidor no departamento, etc.

git Merge

- Na branch shell-script:

```
$ mkdir scripts
$ cd scripts
$ echo 'echo say hello' > hello.sh
$ chmod +x hello.sh
$ ./hello.sh
$ alias say_hello='$PWD/hello.sh'
$ cd ..
$ alias say_hello
```

git Merge

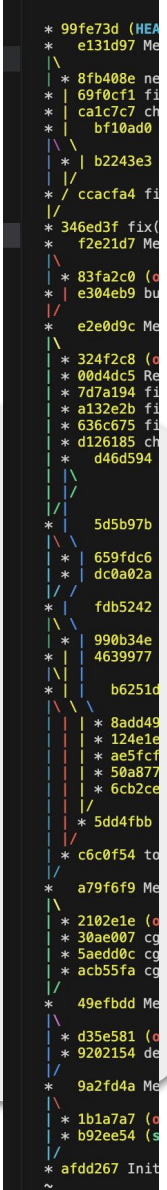
- Na branch shell-script:

```
$ git add .
$ git commit -m "feat: adds say hello script"
$ ls
$ git checkout main
$ ls
$ git merge shell-script
$ ls
$ # o que aconteceu?
```

git Visualização

- Na branch shell-script:

```
$ git log --graph --all
$ git log --graph --all --decorate
$ git log --graph --oneline --all --decorate
```



O que vem depois?

- git pull
- git push
- git revert
- git log

Cadastrem-se no GitHub ou GitLab ou …
Salve e estruture o trabalho de vocês

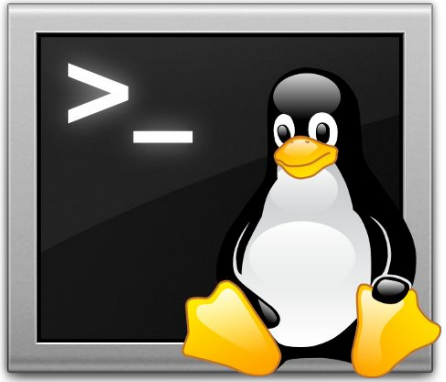
GitHub
https://github.com

GitHub · Change is constant. GitHub keeps you ahead. · GitHub

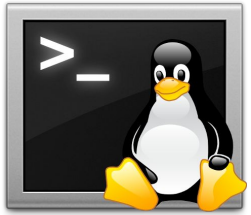
From your first line of code to final deployment, **GitHub** provides AI and automation tools to help you build and ship better software faster. A Copilot chat ...

Shell Script

- Shell Script é um arquivo que contém uma sequência de comandos do terminal, escritos para serem executados automaticamente pelo *shell* (interpretador de comandos do sistema operacional). Ele é usado para automatizar tarefas repetitivas, como executar programas, manipular arquivos, instalar softwares ou **organizar processos no sistema**.



Shell Script: Exercício

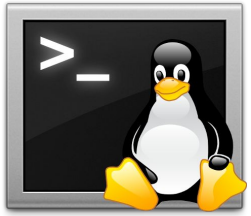


E) Crie uma branch com o script ***gen-prj.sh***:

```
#!/bin/bash

# Check if the project name was provided as an argument
if [ -z "$1" ]; then
    echo "Error: please provide the project name."
    echo "Usage: ./create_prj.sh <project_name>"
    exit 1
fi

PRJ_NAME=$1
BASE_DIR="prj/$PRJ_NAME"

# Check if the project directory already exists
if [ -d "$BASE_DIR" ]; then
    echo "Error: project '$PRJ_NAME' already exists."
    exit 1
fi
```

```
# Create project directory structure
mkdir -p "$BASE_DIR/rtl"
mkdir -p "$BASE_DIR/tb"
mkdir -p "$BASE_DIR/docs"

# Create placeholder files to keep directories in version control
touch "$BASE_DIR/rtl/.empty"
touch "$BASE_DIR/tb/.empty"

# Create an empty README file
touch "$BASE_DIR/docs/README.md"

echo "Project '$PRJ_NAME' successfully created inside prj/"
```

Shell Script: Exercício



- Execute o script

$ chmod +x gen-prj.sh
$ ./gen-prj.sh

***# Q1)** que msg saiu?*
***# Q2)** o que fazer para resolver?*
***# Q3)** o que acontece se executar duas vezes (ou mais) o comando de Q2?*

EDA

- O que é EDA?

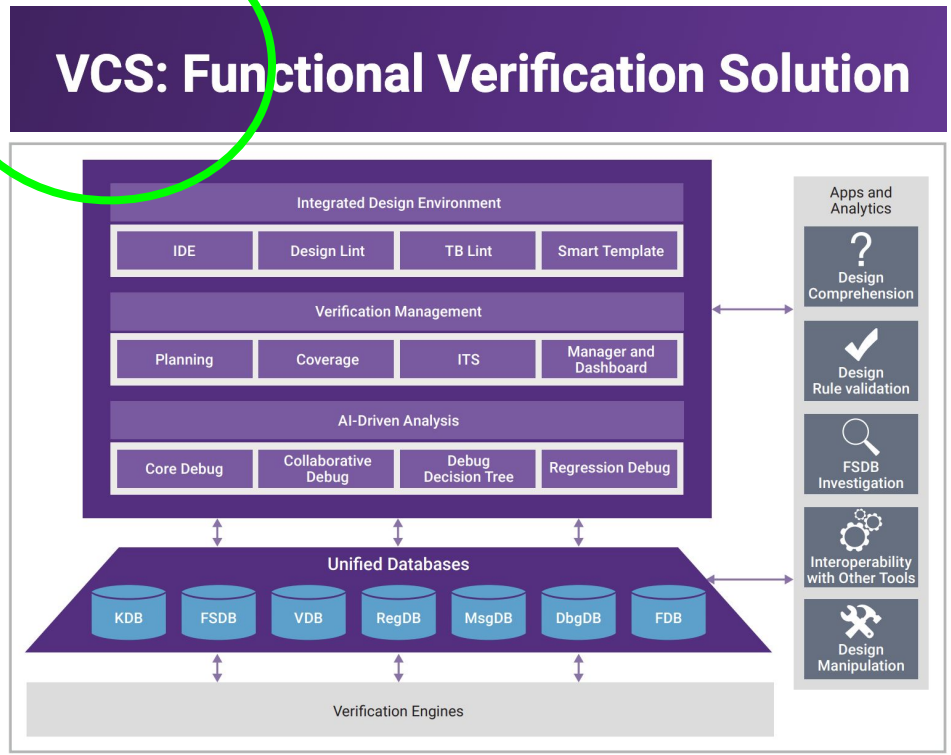
EDA (Electronic Design Automation) é o conjunto de ferramentas de software usadas para projetar, simular e verificar circuitos eletrônicos, principalmente circuitos digitais como chips e FPGAs.

EDA: Design Verification

SYNOPSYS®

Verdi

Innovative debug and verification management platform



EDA: Design Verification

- Crie um projeto chamado hello

```
$ ./gen-prj.sh hello
```

- Acesse a pasta tb

```
$ cd prj/hello/tb
```

- Crie um arquivo chamado tb_top.sv

EDA: Design Verification

- Crie um projeto chamado hello

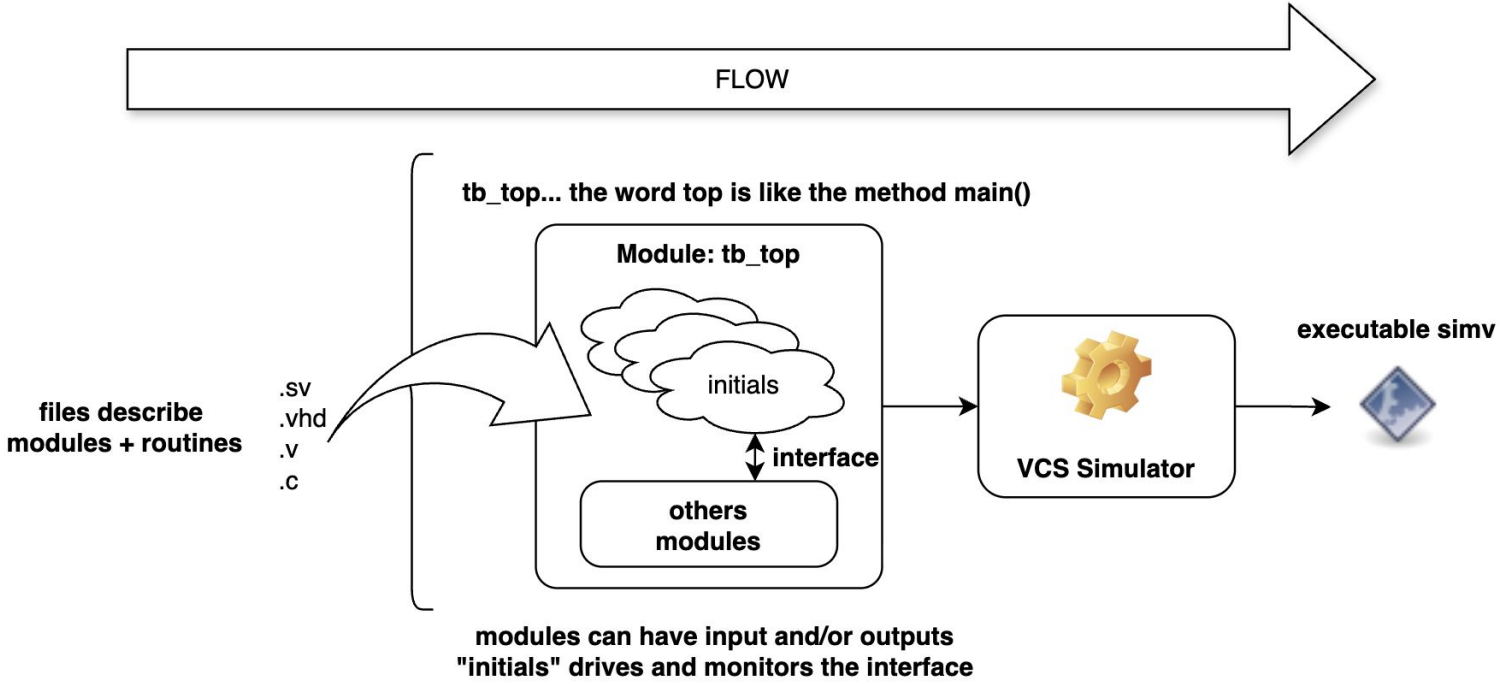
```
module tb_top();
    initial begin
        repeat(10) begin
          $display("Hello, VCS!");
        end
    end
endmodule
```

EDA: Design Verification

- Entendendo um pouco mais…

FILE tb_top.sv:

```
module tb_top();
  initial begin
    $display("Hello, VCS!");
  end
endmodule
```



EDA: Design Verification

- **QUIZZ TIME**

FILE tb_top.sv:

```
module tb_top();
  initial begin
    $display("Hello, VCS!");
  end
endmodule
```

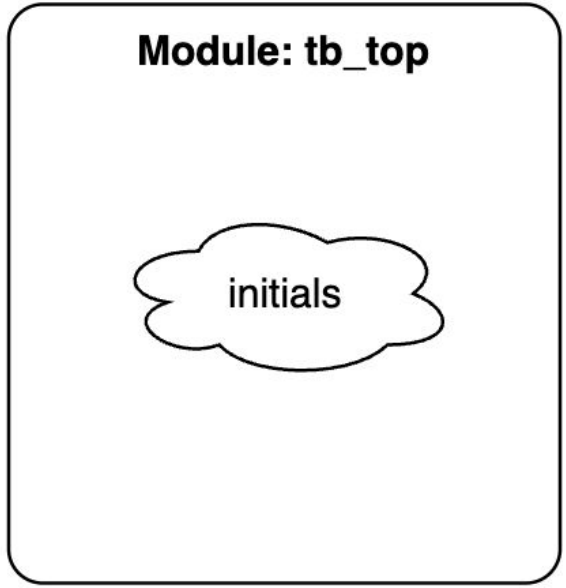
Baseado na figura anterior, como você desenharia esse sistema?

EDA: Design Verification

- **QUIZZ TIME / ANS:**

FILE tb_top.sv:

```
module tb();
  initial begin
    $display("Hello, VCS!");
  end
endmodule
```



EDA: Design Verification

- Habilite e cheque as ferramentas

```
$ source /Tools/snps.sh
$ which vcs
```

EDA: Design Verification

- Crie um arquivo dut.sv na pasta rtl/

```
module dut(
        input logic  a,
        input logic  b,
        output logic c
);
        assign c = a & b;
endmodule
```

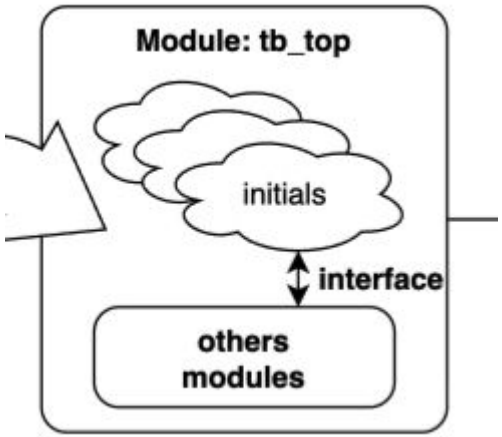
EDA

- Atualize o tb_top:

FILE tb_top.sv:

```
module tb_top();
  initial begin
    $display("Hello, VCS!");
  end
endmodule
```

```
module tb_top();
  logic a,b,c;
  // dut
  dut uu_dut (
    .a(a),
    .b(b),
    .c(c)
  );
  initial begin
    $display("a b | c");
    $display("-----");
    // test all 4 input combinations
    a = 0; b = 0; #1ns;
    $display("%0d %0d | %0d", a, b, c);
    a = 0; b = 1; #1ns;
    $display("%0d %0d | %0d", a, b, c);
    a = 1; b = 0; #1ns;
    $display("%0d %0d | %0d", a, b, c);
    a = 1; b = 1; #1ns;
    $display("%0d %0d | %0d", a, b, c);
    $finish;
  end
endmodule
```

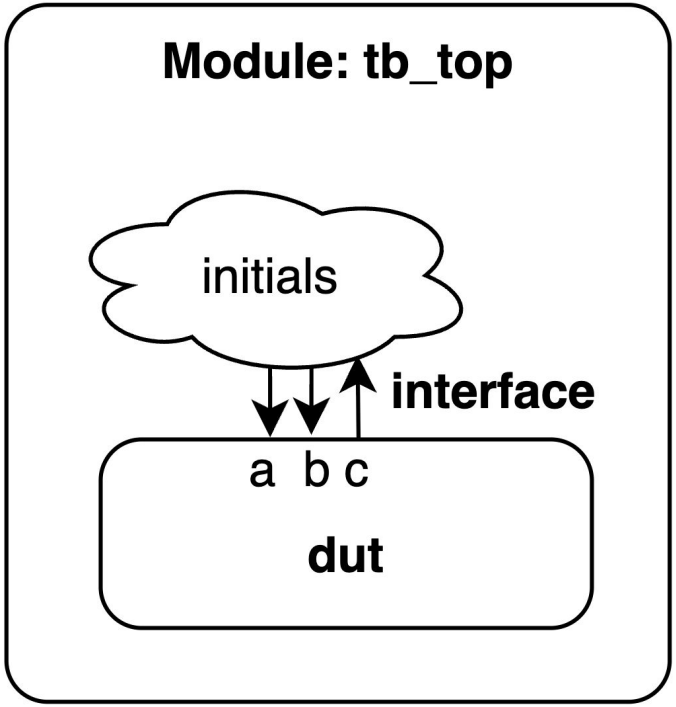


EDA

- **QUIZZ TIME**
  - Como ficaria a figura agora?
  - Identifique o que é interface.

EDA

- QUIZZ TIME
  - Como ficaria a figura agora?
  - Identifique o que é interface.



EDA: Design Verification

- Em tb execute:

```
$ vcs -sverilog ../rtl/dut.sv top_tb.sv
```

- Rode a simulação:

```
$ ./simv
```

- **QUIZZ) O resultado esperado está correto?**

EDA: Design Verification

- No top_tb.sv, adicione após o initial begin:

```
// .. options of dump:
initial begin
  $fsdbDumpfile("waveform.fsdb");
  $fsdbDumpvars("+all");
  $fsdbDumpMDA(); // multi-dimensional array
  $fsdbDumpSVA;

  ...

end
```

EDA: Design Verification

- Passo a passo para carregar o design…

```
$ vcs -sverilog -full64 -debug_access+all -kdb tb.sv ../rtl/dut.sv
```

- Execute o Verdi:

```
$ verdi
```

EDA: Design Verification

- Passo a passo para carregar o design…

```
$ vcs -sverilog -full64 -debug_access+all -kdb tb.sv ../rtl/dut.sv
```

- Execute o Verdi:

```
$ verdi
```

- ou

```
$ simv -gui # or simv -verdi # see the documentation!
```

EDA: Design Verification

- O histórico do verdi fica em verdilog/verdi.cmd

- E1) Fazer o fluxo do verdi.cmd
- E2) ***Reload*** na plataforma

EDA: Design Verification

- Exercício:
  - Crie uma nova branch
  - Crie um novo projeto calc
  - Implemente o DUT (calc4)
  - Implemente o TB
  - Gere formas de onda
  - Faça o merge na main/

```
module calc4 (
   input  logic [7:0] a,
   input  logic [7:0] b,
   input  logic [1:0] op,
   output logic [7:0] result
);

always_comb begin
   case (op)
      2'b00: result = a + b;
      2'b01: result = a - b;
      2'b10: result = a * b;
      2'b11: result = a / b;
      default: result = 8'd0;
   endcase
end

endmodule
```

TCL

- TCL (**Tool Command Language**) é uma linguagem de script simples e flexível muito utilizada em **ferramentas de engenharia eletrônica (EDA)**, como sintetizadores, **simuladores** e ferramentas de FPGA/ASIC. Ela permite automatizar tarefas, controlar fluxos de projeto e executar comandos dentro dessas ferramentas de forma programável. **Em vez de realizar operações manualmente na interface gráfica, o engenheiro pode escrever scripts em TCL para compilar projetos, rodar simulações, configurar parâmetros e organizar o fluxo de desenvolvimento de maneira reproduzível e eficiente.**

TCL

- Crie um arquivo hello.tcl
- Execute:

```
$ ./hello.tcl
```

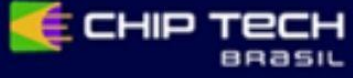
```
#!/usr/bin/tclsh

puts "Hello world" ; #multiple args are not allowed
puts -nonewline "Hello world"
puts "!!!"

# Scalars
set a 5
puts $a

set str "hello five"
puts $str

set "name surname" "Gabriel Villanova"
puts ${name surname}

# Variable Ref
upvar 0 a b;
puts $b
```

Dúvidas CI Expert

Q&A

EXECUÇÃO:

APOIO:

COORDENAÇÃO:

INICIATIVA: